Automatización y Extensión del Entorno: un Comando de Costo, un Hook de Dos Comprobaciones y un Plugin de Revisión

Araceli Castillo

Maestría en Inteligencia Artificial, Atlantis University

MAI 540: Machine Learning

Prof. Kevin A. García Gallardo

15 de octubre de 2026

Automatización y Extensión del Entorno: un Comando de Costo, un Hook de Dos Comprobaciones y un Plugin de Revisión

Esta tarea parte de la aplicación híbrida de la Clase 7 con los once bloques completos: un Random Forest que estima la probabilidad de crédito malo en German Credit (Hofmann, 1994), un modelo de lenguaje que la explica y una capa que verifica el texto. Agregué tres piezas de automatización y para cada una me pregunté qué descuido evita y en qué caso no me protege. Todo está en el repositorio con ocho commits atómicos y se reprodujo desde cero en una carpeta limpia.

El Comando Propio: costo

python tareas.py costo calcula, sobre los 200 casos de prueba, cuántos créditos malos se aprueban y cuántos buenos se rechazan con nueve umbrales de rechazo, y los convierte en costo con la matriz original del dataset: cinco por cada malo aprobado y uno por cada bueno rechazado. Lo repito cada vez que cambio el modelo, porque la Tarea 6.2 me enseñó que la exactitud sola no dice cuánto cuesta equivocarse. La primera ejecución ya cambió una decisión: el umbral 0.50 que usa predict cuesta 194, mientras que 0.30 cuesta 96, con exactitud de 0.700 en lugar de 0.770; aprobar a todos costaría 300. El riesgo que reduce es elegir el modelo o el umbral solo por exactitud. No me protege de una elección optimista: el umbral se elige y se mide con los mismos 200 casos, así que el 96 es optimista, igual que la búsqueda de hiperparámetros de la Tarea 6.2 (Cawley & Talbot, 2010). Tampoco protege si los costos 5 y 1 no representan el negocio real. Dos pruebas nuevas cubren el cálculo, una con un caso pequeño resuelto a mano.

El Hook de Git y su Ampliación

El hook de clase ejecuta pytest antes de cada commit (Chacon & Straub, 2014). Lo comprobé al cambiar prob < 0.30 por prob < 0.99 en verificar_tono: el texto de respaldo dejó de pasar la verificación, la prueba falló y el commit se canceló. Lo amplié con una segunda comprobación: ejecutar verificar_extension.py. La elegí porque pytest no mira la carpeta extension/; un manifiesto roto o un comando sin description pasaría las siete pruebas y solo se

notaría al cargar el plugin. Lo demostré al cambiar description por descripcion: las pruebas pasaron, la segunda comprobación marcó FALTA y el commit se canceló. El hook reduce el riesgo de guardar código que rompe algo que ya está probado. No me protege en tres casos: un commit con git commit --no-verify salta el hook; un clon nuevo del repositorio no trae el hook instalado, porque .git/hooks no se versiona y hay que volver a ejecutar instalar_hook.py; y una regla que ninguna prueba cubre. Lo comprobé: cambié el umbral de explicacion_segura a 0.35, dejé verificar_tono en 0.30 y las siete pruebas pasaron, aunque las dos funciones ya no hablan de las mismas franjas.

La Extensión Estilo Plugin

La extensión costo-de-riesgo tiene su manifiesto, un comando y un agente distintos de los de clase, y pasa verificar_extension.py sin ninguna línea FALTA. El comando /costo-de-riesgo:umbral ejecuta tareas.py costo y pide a Claude cuatro frases con cifras, incluida la advertencia de que el umbral es optimista; al cargarlo con claude --plugin-dir extension respondió con los números correctos y con esa advertencia (Anthropic, 2025). El agente revisor-de-commits lee el git diff y busca lo que el hook deja pasar: umbrales que no coinciden entre funciones, costos cambiados sin razón, pruebas debilitadas y variables sensibles. Con el cambio a 0.35 que pasó las siete pruebas, el agente recomendó no hacer commit, señaló las dos líneas y propuso una prueba con probabilidades en la frontera. La extensión reduce el riesgo de interpretar mal la tabla y de confiar en que «las pruebas pasan» significa «el cambio está bien». No me protege si nadie la invoca, porque no corre sola como el hook, y su revisión depende de un modelo de lenguaje, que puede pasar por alto un problema o señalar uno que no existe: su veredicto orienta, pero no reemplaza una prueba.

Conclusión

Las tres piezas se complementan. El comando mide lo que importa al negocio, el hook bloquea lo que ya está probado y el agente busca lo que todavía no lo está. El siguiente paso es convertir el hallazgo del agente en una prueba: definir los umbrales 0.30 y 0.60 una sola vez y probar sus fronteras, para que el hook cubra lo que hoy depende de una revisión.

Referencias

Anthropic. (2025). *Plugins* [Documentación de Claude Code]. https://code.claude.com/docs/en/plugins

Cawley, G. C., & Talbot, N. L. C. (2010). On over-fitting in model selection and subsequent selection bias in performance evaluation. *Journal of Machine Learning Research, 11*, 2079–2107.

Chacon, S., & Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. https://git-scm.com/book/en/v2/Customizing-Git-Git-Hooks

Hofmann, H. (1994). *Statlog (German credit data)* [Conjunto de datos]. UCI Machine Learning Repository. https://doi.org/10.24432/C5NC77